

Apêndice C

Sistemas de Numeração

Exercícios de Autorrevisão — Resolução Didática

“Computadores armazenam tudo em binário (base 2). Nós humanos pensamos em decimal (base 10). Octal (base 8) e hexadecimal (base 16) são bases auxiliares, escolhidas porque são potências de 2 e oferecem representações mais curtas dos números binários. Este apêndice consolida conversões entre essas quatro bases e a representação de inteiros negativos via complemento de dois.”

1 Exercício C.1 — Lacunas sobre bases

(a) Bases dos sistemas decimal, binário, octal e hexadecimal

10, 2, 8 e 16, respectivamente.

(b) Valor posicional do dígito mais à direita

1 (a base elevada à potência zero, $b^0 = 1$ para qualquer base b).

(c) Valor posicional do dígito imediatamente à esquerda

A base do sistema numérico ($b^1 = b$).

2 Exercício C.2 — Verdadeiro ou Falso

(a) Decimal abrevia binários em grupos de 4 bits

Falso. Quem faz isso é o sistema **hexadecimal**.

Cada dígito hexadecimal corresponde a exatamente 4 bits porque $16 = 2^4$:

$$\underbrace{1111}_F \underbrace{1010}_A \underbrace{1100}_C \underbrace{1110}_E = \text{FACE}_{16}$$

(b) O dígito mais alto é “um a mais que a base”

Falso. O dígito mais alto é **um a menos** que a base.

- Decimal (base 10): dígitos 0 a 9 (mais alto = $9 = 10 - 1$).
- Binário (base 2): dígitos 0 e 1 (mais alto = $1 = 2 - 1$).
- Octal (base 8): dígitos 0 a 7 (mais alto = $7 = 8 - 1$).
- Hexadecimal (base 16): dígitos 0 a F (mais alto = $F = 15 = 16 - 1$).

(c) O dígito mais baixo é “um a menos que a base”

Falso. O dígito mais baixo em qualquer base é sempre **0**.

3 Exercício C.3 — Número de dígitos

MENOS. As representações decimal, octal e hexadecimal contêm menos dígitos do que a binária.

Por quê? Quanto maior a base, mais “informação” cada dígito carrega. Um dígito hexadecimal carrega informação de 4 bits, um dígito decimal carrega cerca de 3.32 bits (já que $\log_2 10 \approx 3.32$), um dígito octal carrega 3 bits e um dígito binário carrega só 1 bit.

Exemplo: o número 255 em quatro bases:

$$255_{10} = 11111111_2 = 377_8 = FF_{16}$$

3 dígitos em decimal, 8 em binário, 3 em octal, 2 em hexadecimal.

4 Exercício C.4 — Representação mais concisa

HEXADECIMAL.

Entre as alternativas (octal, hexadecimal, decimal), a base 16 é a mais concisa por ter o maior conjunto de dígitos. Por isso é usada para representar endereços de memória, códigos de cor RGB (#FFFFFF), valores hash (MD5, SHA-256), etc.

5 Exercício C.5 — Tabela de valores posicionais

Preencha os valores faltantes.

Sistema	P3	P2	P1	P0
Decimal	1000	100	10	1
Hexadecimal	4096	256	16	1
Binário	8	4	2	1
Octal	512	64	8	1

Por quê?

Em cada sistema, os valores posicionais são potências da base, começando por $b^0 = 1$ à direita:

- **Decimal:** $10^0 = 1$, $10^1 = 10$, $10^2 = 100$, $10^3 = 1000$.
- **Hexadecimal:** $16^0 = 1$, $16^1 = 16$, $16^2 = 256$, $16^3 = 4096$.
- **Binário:** $2^0 = 1$, $2^1 = 2$, $2^2 = 4$, $2^3 = 8$.
- **Octal:** $8^0 = 1$, $8^1 = 8$, $8^2 = 64$, $8^3 = 512$.

6 Exercício C.6 — Binário 110101011000 para octal e hex

Para octal: agrupe de 3 em 3 (a partir da direita)

$$\underbrace{110}_6 \underbrace{101}_5 \underbrace{011}_3 \underbrace{000}_0 = \boxed{6530_8}$$

Octal: 6530.

Para hexadecimal: agrupe de 4 em 4 (a partir da direita)

$$\underbrace{1101}_{D=13} \underbrace{0101}_5 \underbrace{1000}_8 = \boxed{D58_{16}}$$

Hexadecimal: D58.

Verificação

$110101011000_2 = 3416_{10}$. Conferindo: $6530_8 = 6 \times 512 + 5 \times 64 + 3 \times 8 + 0 = 3072 + 320 + 24 = 3416$.
E $D58_{16} = 13 \times 256 + 5 \times 16 + 8 = 3328 + 80 + 8 = 3416$.

7 Exercício C.7 — Hexadecimal FACE em binário

Substitua cada dígito hex pelos seus 4 bits:

$$\begin{array}{cccc} \underbrace{F}_{1111} & \underbrace{A}_{1010} & \underbrace{C}_{1100} & \underbrace{E}_{1110} \end{array}$$

Binário: 1111 1010 1100 1110.

8 Exercício C.8 — Octal 7316 em binário

Substitua cada dígito octal pelos seus 3 bits:

$$\begin{array}{cccc} \underbrace{7}_{111} & \underbrace{3}_{011} & \underbrace{1}_{001} & \underbrace{6}_{110} \end{array}$$

Binário: 111 011 001 110.

9 Exercício C.9 — Hexadecimal 4FEC em octal

Dica do livro: primeiro converta para binário, depois para octal.

Etapla 1: hex 4FEC para binário (4 bits por dígito)

$$\begin{array}{cccc} \underbrace{4}_{0100} & \underbrace{F}_{1111} & \underbrace{E}_{1110} & \underbrace{C}_{1100} \end{array}$$

Binário: 0100 1111 1110 1100.

Etapla 2: binário para octal (3 bits por dígito, da direita)

Agrupando de 3 em 3 a partir da direita: 100 111 111 101 100, preenchendo a posição mais à esquerda com zero (a quantidade total de bits é 16, não múltiplo de 3, então adicionamos 2 bits zero à esquerda para totalizar 18, formando 6 grupos de 3 bits).

$$\begin{array}{cccccc} \underbrace{000}_0 & \underbrace{100}_4 & \underbrace{111}_7 & \underbrace{111}_7 & \underbrace{101}_5 & \underbrace{100}_4 \end{array}$$

Octal: 047754 (ou simplesmente 47754).

Verificação

$$4FEC_{16} = 4 \times 4096 + 15 \times 256 + 14 \times 16 + 12 = 16384 + 3840 + 224 + 12 = 20460_{10}.$$

$$E\ 47754_8 = 4 \times 4096 + 7 \times 512 + 7 \times 64 + 5 \times 8 + 4 = 16384 + 3584 + 448 + 40 + 4 = 20460.$$

10 Exercício C.10 — Binário 1101110 em decimal

Multiplique cada dígito pelo seu valor posicional (potências de 2):

Posição	6	5	4	3	2	1	0
Valor pos.	64	32	16	8	4	2	1
Dígito	1	1	0	1	1	1	0
Produto	64	32	0	8	4	2	0

$$64 + 32 + 0 + 8 + 4 + 2 + 0 = \boxed{110_{10}}$$

Decimal: 110.

11 Exercício C.11 — Octal 317 em decimal

Posição	2	1	0
Valor pos.	64	8	1
Dígito	3	1	7
Produto	192	8	7

$$192 + 8 + 7 = \boxed{207_{10}}$$

Decimal: 207.

12 Exercício C.12 — Hexadecimal EFD4 em decimal

Posição	3	2	1	0
Valor pos.	4096	256	16	1
Dígito	E (14)	F (15)	D (13)	4
Produto	57344	3840	208	4

$$57344 + 3840 + 208 + 4 = \boxed{61396_{10}}$$

Decimal: 61396.

13 Exercício C.13 — Decimal 177 em bin, oct, hex

Binário: estratégia das potências subtraídas

Encontramos a maior potência de 2 menor ou igual a 177 ($128 = 2^7$) e seguimos:

Valor pos. 1	128	64	32	16	8	4	2
Cabe? SIM, sobra 0	SIM, sobra 49	NÃO	SIM, sobra 17	SIM, sobra 1	NÃO	NÃO	NÃO
Dígito 1	1	0	1	1	0	0	0

$$177 = 128 + 32 + 16 + 1$$

Binário: 10110001.

Octal: usando valores 64, 8, 1

$$177 = 2 \times 64 + 6 \times 8 + 1 = 128 + 48 + 1$$

Octal: 261.

Hexadecimal: usando valores 16 e 1

$$177 = 11 \times 16 + 1 = 176 + 1$$

Lembrando que $11 = B_{16}$:

Hexadecimal: B1.

Verificação cruzada

Conferindo se as 3 representações batem:

- Binário agrupado em 3: $10110001 = \underbrace{010}_2 \underbrace{110}_6 \underbrace{001}_1 = 261_8$.
- Binário agrupado em 4: $10110001 = \underbrace{1011}_B \underbrace{0001}_1 = B1_{16}$.

14 Exercício C.14 — Complemento de 1 e 2 de 417

Etapa 1: 417 em binário

$$417 = 256 + 128 + 32 + 1:$$

Valor pos.	256	128	64	32	16	8	4	2
1								
Dígito	1	1	0	1	0	0	0	0
1								

Binário: 110100001 (9 bits).

Etapa 2: complemento de UM

Inverte cada bit (0 vira 1 e vice-versa):

$$\begin{array}{r|cccccccc} 417 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ \hline \sim 417 \text{ (C1)} & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 \end{array}$$

Complemento de UM: 001011110.

Etapa 3: complemento de DOIS

Some 1 ao complemento de UM:

$$001011110 + 1 = 001011111$$

Complemento de DOIS: 001011111.

Verificação: $n + (-n) = 0$ em complemento de dois

$$\begin{array}{r} 110100001 \\ + 001011111 \\ \hline (1) 000000000 \end{array}$$

O “1” transbordado fora dos 9 bits é *descartado* pela aritmética de complemento de dois. O resultado é 000000000, ou seja, **zero**.

15 Exercício C.15 — Soma de um número com seu C2

ZERO.

Justificativa

Em complemento de dois com n bits, define-se $-x = 2^n - x$ (módulo 2^n). Logo:

$$x + (-x) = x + (2^n - x) = 2^n \equiv 0 \pmod{2^n}$$

O bit que “vaza” na posição $n + 1$ é simplesmente descartado pela aritmética modular.

Observação

É exatamente este comportamento que torna a aritmética em complemento de dois tão elegante: a mesma circuiteria que faz adição também faz subtração (basta adicionar o complemento). Por isso processadores modernos usam C2 universalmente para representar inteiros com sinal.

Síntese conceitual

A autorrevisão consolidou as técnicas centrais:

- **Bases:** binária (2), octal (8), decimal (10), hexadecimal (16). Cada uma com b dígitos (de 0 a $b - 1$).
 - **Conversões entre potências de 2:** binário $\leftrightarrow$ octal usa grupos de 3 bits; binário $\leftrightarrow$ hex usa grupos de 4 bits.
 - **Conversão para decimal:** multiplique cada dígito pelo seu valor posicional e some.
 - **Conversão do decimal:** extraia repetidamente o resto da divisão pela base (ou monte potências subtraindo).
 - **Complemento de UM:** inverter todos os bits.
 - **Complemento de DOIS:** $C1 + 1$. Representação padrão de inteiros negativos.
 - **Propriedade fundamental:** $x + (-x) = 0$ em C2 (com transbordamento descartado).
-